

Firestore Messaging Lifecycle Package — Full AI Development Specification

1. Package Vision

Package Name

`firebase_messaging_manager`

Core Goal

Build a production-grade Flutter package that simplifies Firestore push notification handling across Android and iOS.

The package should:

- handle notification lifecycle events
- prevent duplicate notifications
- simplify Firestore Messaging setup
- simplify local notification integration
- normalize notification events
- provide clean callbacks for app developers
- reduce Firestore notification boilerplate

This package is intended to wrap and orchestrate:

- `firebase_messaging`
- `flutter_local_notifications`

This package is NOT intended to replace Firestore Messaging. It is a higher-level abstraction layer.

2. Main Problem Statement

Flutter developers commonly face:

- duplicate notifications
- foreground/background notification inconsistencies
- terminated app handling complexity
- iOS/Android behavior differences
- confusing notification lifecycle APIs
- difficult local notification orchestration
- notification tap routing complexity
- notification permission handling complexity

The package must solve these problems cleanly.

3. Main Features (MVP)

3.1 Notification Lifecycle Management

The package must support:

Foreground Notifications

When app is open.

Background Notifications

When app is in background.

Terminated Notifications

When app is killed and opened from notification.

Notification Tap Handling

User taps notification.

Initial Notification Handling

App launched from terminated state.

3.2 Duplicate Notification Prevention

This is the MOST IMPORTANT feature.

The package must:

- detect duplicate notifications
- avoid duplicate local notifications
- optionally dedupe using messageId
- optionally dedupe using custom payload key
- provide configurable dedupe duration

Possible strategies:

```
DuplicateStrategy.none  
DuplicateStrategy.messageId  
DuplicateStrategy.customKey
```

Configurable:

```
dedupeDuration: Duration(seconds: 10)
```

3.3 Local Notification Orchestration

The package must:

- integrate with flutter_local_notifications
- show local notifications when appropriate
- avoid duplicate notifications
- support Android notification channels
- support notification sounds
- support tap callbacks

3.4 Notification Permission Handling

The package must:

- request permissions on Android
- request permissions on iOS
- expose permission status
- provide helper methods

3.5 Unified Notification Event Model

Developers should receive ONE normalized event object.

Example:

```
class NotificationEvent {  
  final String? id;  
  final String? title;  
  final String? body;  
  final Map<String, dynamic> data;  
  final NotificationAppState appState;  
  final bool wasTapped;  
  final RemoteMessage? rawMessage;  
}
```

3.6 Clean Callback API

The package must expose a clean API.

Example:

```
await FirebaseMessagingLifecycle.initialize(  
  onForegroundMessage: (event) {},  
  onNotificationTap: (event) {},  
  onInitialNotification: (event) {},  
);
```

4. Package Philosophy

The package MUST:

- prioritize simplicity
- reduce setup complexity
- reduce boilerplate
- be production safe
- work predictably
- avoid overengineering
- provide beginner-friendly APIs
- provide strong documentation

The package MUST NOT:

- become bloated
- attempt to become a full notification platform
- include unnecessary analytics
- include backend services
- include custom server implementation

5. Recommended Tech Stack

Flutter SDK

Stable Flutter version.

Dependencies

Required

- firebase_messaging
- flutter_local_notifications

Optional

- collection

- equatable
-

6. Architecture Rules

6.1 Clean Internal Structure

Suggested structure:

```
lib/  
  firebase_messaging_lifecycle.dart  
  
src/  
  models/  
  enums/  
  services/  
  handlers/  
  utilities/  
  callbacks/
```

6.2 Public API Rules

ONLY expose:

- essential classes
- essential models
- configuration objects
- enums

DO NOT expose internal implementation.

6.3 Singleton Manager

Use singleton pattern.

Example:

```
FirebaseMessagingLifecycle.instance
```

7. Required Enums

NotificationAppState

```
enum NotificationAppState {  
    foreground,  
    background,  
    terminated,  
}
```

DuplicateStrategy

```
enum DuplicateStrategy {  
    none,  
    messageId,  
    customKey,  
}
```

8. Main Configuration Object

```
class FirebaseMessagingLifecycleConfig {  
    final bool requestPermission;  
    final bool showForegroundNotifications;  
    final bool showBackgroundNotifications;  
    final DuplicateStrategy duplicateStrategy;  
    final Duration dedupeDuration;  
    final String androidChannelId;  
    final String androidChannelName;  
}
```

9. Public Initialization API

Example desired API:

```
await FirebaseMessagingLifecycle.initialize(  
    config: FirebaseMessagingLifecycleConfig(...),  
    onForegroundMessage: (event) {},  
    onNotificationTap: (event) {},
```

```
onInitialNotification: (event) {},  
);
```

10. Core Notification Flow

Foreground

- Receive Firebase message
- Normalize payload
- Check duplicates
- Show local notification if enabled
- Trigger callback

Background

- Receive Firebase message
- Handle background tap
- Normalize payload
- Trigger callback

Terminated

- Detect initial notification
- Normalize payload
- Trigger callback

11. Duplicate Prevention Logic

This is a CRITICAL requirement.

The package must:

- store recently handled notification IDs
- ignore repeated IDs
- support configurable dedupe duration

Suggested approach:

```
final Map<String, DateTime> handledNotifications = {};
```

Remove expired entries periodically.

12. Android Requirements

The package must:

- support Android notification channels
- support Android 13+ permissions
- support foreground notifications
- support notification icons
- support notification sounds

13. iOS Requirements

The package must:

- support iOS permissions
- support foreground presentation options
- avoid duplicate foreground notifications
- support notification tap handling

14. Example App Requirements

The example app MUST:

- be production quality
- demonstrate all notification states
- demonstrate duplicate prevention
- demonstrate local notification handling
- demonstrate tap navigation

The example app is EXTREMELY important.

15. Documentation Requirements

The README must include:

Installation

Firestore setup

Android setup

iOS setup

Initialization example

Notification callbacks example

Duplicate prevention explanation

Foreground/background explanation

Common issues

FAQ

16. Pub.dev Optimization

The package description should include:

- firebase messaging
- flutter notifications
- local notifications
- duplicate prevention
- lifecycle management
- notification routing
- push notifications

Suggested pub.dev description:

A production-safe Firebase Messaging lifecycle manager for Flutter with duplicate notification prevention, local notification orchestration, foreground/background handling, notification tap routing, and clean callbacks.

17. Coding Standards

Rules

- use meaningful names
 - avoid overengineering
 - avoid unnecessary abstraction
 - use immutable models
 - use null safety
 - avoid deep nesting
 - keep APIs intuitive
 - keep public APIs minimal
-

18. Error Handling Rules

The package must:

- fail gracefully
 - never crash app unnecessarily
 - provide meaningful debug logs
 - validate configuration
-

19. Logging Rules

Provide optional debug logs:

```
enableLogs: true
```

Logs should include:

- notification received
 - duplicate ignored
 - local notification shown
 - notification tapped
-

20. Future Roadmap (NOT MVP)

DO NOT implement initially.

Possible future features:

- action buttons

- grouped notifications
 - scheduled notifications
 - topic subscription helpers
 - advanced analytics
 - navigation integration
 - notification history
 - notification persistence
-

21. What Makes This Package Valuable

The package value is NOT:

- sending notifications
- replacing Firebase

The value IS:

- simplified lifecycle handling
 - duplicate prevention
 - production-safe orchestration
 - cleaner developer experience
-

22. AI Development Instructions

The AI developing this package must:

- write production-grade Flutter code
- follow clean architecture principles
- avoid unnecessary complexity
- prioritize developer experience
- prioritize reliability
- prioritize maintainability
- create reusable utilities
- create readable documentation
- ensure Android/iOS compatibility

The AI must NOT:

- introduce unnecessary dependencies
 - create overcomplicated abstractions
 - use poor naming
 - generate undocumented APIs
 - generate dead code
-

23. Development Order

Phase 1

- package setup
- basic initialization
- foreground handling
- local notification integration

Phase 2

- notification tap handling
- terminated app handling
- background handling

Phase 3

- duplicate prevention
- configuration system
- normalized event model

Phase 4

- example app
 - documentation
 - README optimization
 - pub.dev preparation
-

24. Success Criteria

The package is successful if developers can:

- integrate notifications quickly
- avoid duplicate notifications
- handle lifecycle events easily
- reduce Firebase notification boilerplate
- navigate from notifications cleanly

with minimal setup.

25. Competitive Inspiration

The package should take inspiration from:

`push_notification_manger`

while significantly improving:

- API cleanliness
- duplicate notification prevention
- lifecycle orchestration
- documentation quality
- developer experience
- foreground/background handling clarity
- terminated app handling
- typed callbacks
- normalized notification event models
- production readiness
- Android/iOS consistency

The package should study strengths and weaknesses of existing notification orchestration packages and provide a cleaner, safer, more production-oriented abstraction.

26. Final Vision

The package should become:

The easiest and safest way to manage Firebase Messaging lifecycle events in Flutter.

The package should focus on:

- simplicity
- reliability
- production readiness
- lifecycle orchestration
- duplicate prevention
- minimal setup
- developer experience

rather than feature bloat.